

CHECKPOINTS



Full Brief

①

Checkpoint 1

②

Checkpoint 2

③

Checkpoint 3



Advanced (Optional)

ON THIS PAGE



Checkpoints



Scenario



Requirements



Final Deliverables



Grading Rubric



Checkpoints

This project is graded progressively across **three bi-weekly checkpoints**, one repository built up incrementally from Week 2 to Week 6. Each checkpoint has its own scoped deliverables and grading rubric; together they cover the full project described below.

WEEK 2



Checkpoint 1: Database Design

Database schema design, sample data, SQL querying

WEEK 4



Checkpoint 2: Flask & SQLAlchemy Layer

Flask + SQLAlchemy models, migrations, relationships

WEEK 6



Checkpoint 3: Complete & Deployed

Full REST API, testing, and deployment



A Week 7 1-on-1 session reviews the assignment as a whole — no separate submission is needed for it. This document describes the **final state** of the project; see each checkpoint brief above for what's due and how it's graded at that stage.



Scenario

You have been hired as a backend developer to build a secure and scalable online store API for a fictional e-commerce platform named **RevoShop**. The API will be consumed by frontend applications that interact with the store's core data, including products, categories, orders, and users.

Expected output: Your application should be similar in functionality and design to a [Pet Store API](#).



Important Notice

This assignment is designed to strengthen your backend development skills and deepen your understanding of building scalable APIs with Python, Flask, and SQLAlchemy. While AI tools may be used to assist in exploration and idea generation, it

[← Back to Syllabus](#)

Module 2 Assignment: RevoShop API — Backend Development Project Milestone

[\[→ Logout\]](#)

Remember, your journey as a developer is about both the process and the final product. Embrace the challenges, ask questions, and make the most of every opportunity to learn and grow!

Requirements

You are required to implement the following endpoints.



User Module

POST /users Register a new user

POST /auth/login Login



Product Module

POST /products Create a new product

GET /products List all products

GET /products/<id> Get a specific product

PUT /products/<id> Update a product

DELETE /products/<id> Delete a product (blocked if active orders exist)

Category Module

POST /categories Create a new category

GET /categories List all categories

GET /categories/<id> Get a specific category along with its products

PUT /categories/<id> Update a category

DELETE /categories/<id> Delete a category

Order Module

POST /orders Place a new order linked to the logged-in user

GET /orders List all orders for the current user

GET /orders/<id> View a specific order with its order items and product details

DELETE /orders/<id> Delete an order



"Logged-in user" note: Module 2 does not cover session or token-based authentication (JWT is optional/exploratory only). It's enough for the client to send `user_id` in the request body/params for `POST /orders` and `GET /orders` — the same pattern already used for every other `POST` / `PUT` endpoint.



Final Deliverables





This is the complete, end-state deliverable the project builds toward by Checkpoint 3 (Week 6).

1. Deployed Database — A live, hosted PostgreSQL instance connected to your deployed Flask API, with:

- All migrations successfully applied via `flask db upgrade`.
- All tables (`users`, `products`, `categories`, `orders`, `order_items`) visible and correctly structured.
- All constraints, foreign keys, and relationships verified as working in the production environment.

2. Deployed API — A live, functional deployment of your Flask backend, accessible via a public URL, with all CRUD endpoints for products, categories, orders, and users reachable and returning correct JSON responses with appropriate HTTP status codes.

3. Source Code Repository — A GitHub repository containing the complete Flask project, with:

- A clean folder structure separating models, routes, and configuration.
- A proper `.gitignore` excluding `.env`, `__pycache__`, and virtual environment folders.
- A meaningful commit history reflecting your development progress across database setup, model creation, API development, testing, and deployment.

4. Project Documentation (`README.md`) including:

- **Overview** — a brief explanation of what RevoShop does: an online store API that manages users, products, categories, orders, and order items through a RESTful interface backed by PostgreSQL.
- **Features implemented** — full CRUD for products, categories, and orders; the many-to-many relationship between orders and products through `order_items`; data validation; error handling with `try/except`; and the deletion guard that blocks removing a product with active orders still linked to it.
- **Technologies used** — Flask, SQLAlchemy, Flask-Migrate, PostgreSQL, pgAdmin, pytest, Locust, python-dotenv, gunicorn, and your chosen deployment platform.
- **How to run the project locally** — step-by-step setup: clone the repository, create and activate a virtual environment, install dependencies from `requirements.txt`, configure `.env` using the provided `.env.example` as a reference, and run `flask db upgrade` to apply all migrations before starting the app.
- **Screenshots** — Postman screenshots demonstrating at least one working request for each HTTP method (GET, POST, PUT, DELETE); a pgAdmin view confirming the correct table structure and relationships; a screenshot of the hosted database instance showing all tables live in production; and optionally, Locust dashboard screenshots showing response times and failure rates under simulated load.



Make sure your deployed database, deployed API, and repository links are all accessible before submitting — if a reviewer cannot reach them, your submission cannot be graded.



Grading Rubric

You are graded by **checklist**: each criterion is either met or not met. Aim to satisfy every criterion.

PostgreSQL Setup & Schema Design

Successfully installs PostgreSQL locally, sets a password for the `postgres` superuser, and verifies the installation.

Successfully creates a database named `revoshop_db` through the pgAdmin GUI and verifies it appears in the server tree.

Includes appropriate columns for each table with correct data types.

Primary keys are defined for each table and follow a consistent naming convention.

Column names are descriptive, consistent in format (e.g., `snake_case`), and accurately reflect the data they store.

Correctly creates the `order_items` junction table, with `order_id` and `product_id` defined as foreign keys referencing `orders` and `products`.

Sample Data & Querying

Correctly uses `INSERT INTO` syntax to add sample data with all required fields populated.

Inserted data includes realistic and varied sample values (e.g., multiple users and products, not just one row).

Combines `WHERE`, `ORDER BY`, and `LIMIT` together in at least one query to demonstrate understanding of clause ordering.

Version Control & Documentation

Repository has a clean structure and a commit history that reflects incremental progress.



Includes a proper `.gitignore`.

`README.md` clearly documents how to set up and load the database locally.

Flask & SQLAlchemy Foundations

Defines a route for listing all products that returns the full hardcoded list as a proper JSON response using `jsonify()`.

Defines a route for retrieving a single product by ID that correctly filters the hardcoded list and returns the matching product as a JSON response.

Hardcoded product data is structured as a list of dictionaries with consistent and meaningful keys (e.g., `id`, `name`, `price`).

Correctly configures `SQLALCHEMY_DATABASE_URI` in Flask config using the proper PostgreSQL connection string format (`postgresql://user:password@host/dbname`).

Successfully initializes `db = SQLAlchemy(app)` and verifies the connection is live by running the app without errors.

Correctly defines the `User` model with all required fields: `id`, `username`, `email` (marked as unique), `password_hash`, and `created_at` with a default current timestamp.

Models, Migrations & Relationships

Implements a register route that accepts user input, creates a new `User` instance, and saves it using `db.session.add()` and `db.session.commit()`.

Implements a retrieve route that correctly fetches and returns a user by ID, handling the case where the user is not found.

Correctly adds a `role` column to the `User` model, generates a migration with `flask db migrate`, and applies it with `flask db upgrade`, verifying the column in pgAdmin without affecting existing rows.

Correctly defines the association table using `db.Table()` with the appropriate foreign key columns referencing both the `orders` and `products` tables.

Successfully runs the migration and applies it without errors, with the association table verified as created in pgAdmin.

Inserts sample data linking at least one order to multiple products through the association table, demonstrating a working many-to-many relationship.



API Development

Correctly builds two GET endpoints (`GET /products` and `GET /products/<id>`) returning proper JSON responses using `jsonify()` and a `to_dict()` method on the model.

Correctly implements `GET /orders` and `GET /orders/<id>` following the same pattern, returning the full list and a single order respectively.

Correctly implements `POST /products` to create a new product.

Correctly implements `PUT /products/<id>` to update an existing product.

Correctly implements `POST /auth/login` to log in.

Correctly implements `POST /orders` to create a new order.

Correctly implements `PUT /orders/<id>` to update an existing order.

Correctly implements validation in the `POST /products` endpoint.

Correctly implements validation in the `PUT /products/<id>` endpoint.

Correctly implements `DELETE /orders/<id>`.

Correctly implements `DELETE /products/<id>` (blocked if active orders exist).

Correctly implements `GET /categories`.

Correctly implements `GET /categories/<id>`.

Correctly implements `POST /categories`.

Correctly implements `PUT /categories/<id>`.

Correctly implements `DELETE /categories/<id>`.

Quality Assurance & Deployment

Writes test cases for all CRUD endpoints of Category (GET all, GET by id, POST, PUT, DELETE) covering both happy path and error cases for each.

Happy path test cases verify that each endpoint returns the correct status code and expected response data when valid input is provided.



Error case test cases verify that each endpoint returns the appropriate error status code and meaningful message when invalid or missing data is provided.

Correctly extends the Locust file to simulate a sequential user journey: GET all products, GET a single product by ID, POST a new order, and GET the created order.

Successfully runs the simulation starting with 50 users and gradually increasing to 200 users without configuration errors.

Correctly moves all sensitive config (database URL, secret key, debug mode) out of `app.py` into a `.env` file using `python-dotenv` and `os.getenv()`, with the app still running correctly.

Creates a `.env.example` with placeholder values that is safe to commit, and correctly adds `.env` to `.gitignore`.

Tests the live API in Postman using the public URL — creates a product, fetches it, updates it, and deletes it — confirming full CRUD works in production.



Tips for Success

- **Design the schema first.** Get your tables, primary keys, and foreign keys right before writing any Flask code.
- **Build incrementally.** Get one model and one CRUD route fully working end-to-end (including a migration) before moving to the next.
- **Test as you go.** Write pytest cases for an endpoint right after you build it, not all at once at the end.
- **Keep secrets out of source control from day one.** Set up `.env` and `.gitignore` early rather than retrofitting them before deployment.
- **Deploy early, deploy often.** Don't wait until everything is "done" locally to attempt your first cloud deployment — surface environment issues early.



This project brings together everything from Module 2 — database design, Flask, SQLAlchemy, testing, and deployment — into one real-world backend system. Good luck!

